

MBAAt Club Datathon 2026 — Pre-Event Manual

MBAAt Club

Welcome

This is the **pre-event manual** for the MBAT Club Datathon 2026 (Econometrics and Business Statistics), held Monday 21 to Tuesday 22 September at the Learning and Teaching Building, Clayton campus.

It covers everything you can (and should) set up and read **before** you arrive: Git/GitHub, the Quarto report and slides workflow, and further reading for text mining.

Before you arrive

- Install R, RStudio (or Positron/VS Code with the R extension), Quarto, and GitHub Desktop.
- Create a GitHub account if you don't already have one, and share your username with organisers, they create and share your team's repository, so you just need to clone it (Section).
- Skim Section and Section ahead of time. Day 1 morning is registration, opening remarks, and the dataset introduction, and the working session doesn't start until 10:30 AM, so setup issues are much cheaper to fix beforehand than mid-competition.

Git + GitHub Desktop

Why this matters

Your submission is a public GitHub repository. A team that can't reliably commit and push loses time to lost work, overwritten files, or a repo that isn't actually up to date at the deadline.

The daily loop

Everything below is a variation on this:

1. **Pull** the latest changes.
2. Do some work.
3. **Commit** with a short message.
4. **Push** so your team can see it.

Install (before the event)

1. Install [GitHub Desktop](#).
2. Sign in with a GitHub account (create one first if needed).
3. Let it install Git automatically if prompted.
4. Make sure organisers have your GitHub username so you can be added to your team's repository.

Get your team's repository

Organisers create and share one repository per team, already set up from a starter template, you don't create anything (the starter template's layout is covered in the full manual you'll get on the day).

1. Get the repo link from organisers.

2. Every team member: GitHub Desktop → **File** → **Clone Repository** → paste the URL → **Clone**.
3. Before the working session starts, confirm you can push: make a trivial commit (e.g. add your name to the README) and push it.

Work session workflow

1. Click **Fetch origin**, then **Pull origin** if you're behind do this *before* you edit anything.
2. Edit your files as normal.
3. Check the **Changes** tab, untick anything you didn't mean to include.
4. Write a short summary → **Commit to main**.
5. Click **Push origin**.
6. Repeat: pull → work → commit → push.

Commit often, every working chunk of progress, not just once at the end of the day.

Avoid stepping on your teammates

- Split work by file/section so two people aren't editing the same lines at once.
- Pull before you start a session; push before you take a break.
- Say out loud (or in chat) before editing a shared file, e.g. the report.

Fix a merge conflict

1. Open the file GitHub Desktop flags as conflicted.
2. Find the marked sections:

```
<<<<<<< HEAD
your version
=====
their version
>>>>>>> branch-name
```
3. Edit the file to keep the correct content (or merge both).
4. Delete the <<<<<<<, =====, >>>>>>> marker lines.
5. Save, then commit the resolved file as usual.
6. Stuck for more than a few minutes? Ask the help desk.

Never commit these

- **Credentials:** API keys, connection strings, `.Renviron` contents. Keep them in a local `.Renviron` file, listed in `.gitignore`.
- **Large raw data dumps:** pull the data fresh at analysis time instead of committing it.
- **Cache/temp folders:** `.quarto/`, `*_cache/`. (`_docs/` is the exception, it *does* need to be pushed, since it's your submitted report and slides; see the full manual's starter template chapter.)

If a credential is ever pushed by accident, please notify the staff, deleting the file does not remove it from the repo's history.

Prefer the command line?

```
git clone <repo-url>
git status
git add <file>          # or `git add .`
git commit -m "message"
git pull
git push
```

Same repository either way, teammates can mix GitHub Desktop and the terminal freely.

Before you submit (Day 2, 11:00 AM deadline)

- ☐ The final commit is pushed, an unpushed commit does not count as submitted
- ☐ No credentials appear anywhere in the repo, including old commits
- ☐ Both the rendered HTML and PDF reports open correctly from a fresh clone
- ☐ The presentation slides (Section) are finished and rendered, required from every team, not just finalists

Quarto

i Why this matters

Your submission requires both a reproducible report and a set of presentation slides, Quarto builds both from the same kind of source file. It runs your R code and inserts the real output every time you render, so you never copy-paste a number or a plot in by hand.

The starter template you get on the day already includes both: `docs/report.qmd` for the written report and `docs/slides.qmd` for presentation slides.

Anatomy of a .qmd file

```
---
title: "Team Name - Datathon Submission"
author: "Team Name"
format: html
---
```

```
# Introduction
```

```
We analyse the data to answer...
```

```
```{r}
library(tidyverse)
summary(cars)
```
```

- Text outside code chunks is normal Markdown.
- Anything inside ````{r} ```` blocks runs at render time.
- Add `#| echo: false` inside a chunk to keep the output but hide the code — useful for staying within a word limit.

Render your report

- RStudio/Positron: click **Render**, or press Cmd/Ctrl+Shift+K.
- Command line:

```
quarto render report.qmd      # render using the format in the YAML
quarto render report.qmd --to pdf # force a specific format
quarto preview report.qmd     # live preview, reloads on save
```

Use `quarto preview` while you write, it catches broken chunks immediately instead of at the end.

Set your output format

Both HTML and PDF are required for the report, list both formats in the YAML and Quarto renders one file for each:

```
format:
  html:
    toc: true
    code-fold: true # readers can expand/collapse code blocks
  pdf:
    documentclass: article
```

```
quarto render report.qmd # produces report.html and report.pdf
```

The report has a **3,000-word limit**, excluding code, tables, figure captions, and references.

Build presentation slides (required from every team)

A finished slide deck is part of your submission, not an optional extra, every team builds one, even though only the 5 finalists actually present it live (2:15–3:30 PM, Day 2: 10 min talk + 5 min Q&A each). Treat it with the same care as the report: it's judged on Professional Communication just like everything else you submit.

```
---
title: "Team Name - Datathon Presentation"
format: revealjs
---
```

1. Start each new slide with a `##` heading.
2. Write R chunks exactly as you would in the report, plots and tables render live into the deck.
3. Render the same way: **Render** button, `Cmd/Ctrl+Shift+K`, or `quarto render slides.qmd`.
4. Open the resulting `.html` in a browser; press `f` for fullscreen, use arrow keys or space to advance.

```
## Introduction

The question we investigated...

## Our Approach

```{r}
ggplot(data, aes(x, y)) + geom_point()
```

## Results

Key finding...
```

Set up PDF rendering (before the event)

Since PDF is a required output, don't leave this until Day 2. PDF needs LaTeX, install it once, ahead of time:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Render both formats and open both files before the deadline, a report that renders fine to HTML can still fail on PDF (e.g. an image or table that doesn't fit the page).

Keep the report reproducible and inside the word limit

1. Render early and often, don't wait until the deadline to find a broken chunk.
2. Decide how much code you want visible in the final report, the `echo`, `warning`, and `message` options (set per chunk, or globally under `execute:` in the YAML) control this. There's no required setup here; choose whatever makes your report read cleanly.

3. Cache slow steps (a slow data pull, a model fit) with `#| cache: true` on that chunk.
4. Use `knitr::kable()` or `gt` for tables, and pipe large printed output through `head()`, raw tibble/model printouts can quietly blow out your word count.
5. Keep the raw `.qmd` in your GitHub repo, not just the rendered file, the repo is what gets checked for reproducibility.

Learn more

- `<starttr.numbat.space>`, the club's starting point for Quarto.
- <https://quarto.org/docs/guide/> official docs, every format and option.
- `quarto --help` on the command line, or `?quarto::quarto_render` in R.

Stuck on a YAML typo for more than a few minutes? Ask the help desk.

Resources for tidytext (Text Mining with R)

i Why this matters

The dataset is mostly free text. This chapter is a starting point for tokenising, TF-IDF, sentiment, and word co-occurrence, and where to go for more depth or techniques beyond what's covered during the event.

The core reference

- [Text Mining with R](#) by Julia Silge and David Robinson (free, by the tidytext package authors). Relevant chapters:
 - Ch. 1 — the tidy text format (`unnest_tokens()`)
 - Ch. 2 — sentiment analysis
 - Ch. 3 — word/document frequency (tf-idf)
 - Ch. 4 — n-grams and word correlations
 - Ch. 6 — topic modeling
- `?tidytext::unnest_tokens`, `?tidytext::bind_tf_idf`, or `vignette(package = "tidytext")` for the function reference.

If you want to find latent themes: topic modeling (LDA)

```
library(topicmodels)

dtm <- tokens_clean |>
  count(report_number, word) |>
  cast_dtm(report_number, word, n)

lda_model <- LDA(dtm, k = 5, control = list(seed = 1234))

tidy(lda_model, matrix = "beta") |> # word probabilities per topic
```

```
group_by(topic) |>
slice_max(beta, n = 10)
```

`tidy()` converts the model back into a tidy data frame so the rest of your `dplyr/ggplot2` pipeline still applies. See Text Mining with R, Ch. 6.

If word variants are diluting your counts: stemming

```
library(SnowballC)

tokens_clean <- tokens_clean |>
  mutate(word_stem = wordStem(word))
```

Use deliberately, not automatically, stemming can hurt readability in a report meant for human judges.

If you want more corpus-linguistics tooling: quanteda

[quanteda](#) is an alternative to `tidytext`, with built-in keyword-in-context (`kwic()`) and document-feature matrix tools. Pick one framework and stay consistent through your analysis.

Visualizing results

- `ggplot2::geom_col()` / `geom_point()` for frequency and tf-idf charts.
- `wordcloud2` for a quick visual summary, pair with an actual frequency table, since word clouds are weak for precise comparisons.
- `ggraph` + `igraph` on `widyr::pairwise_cor()` output for word-correlation networks.

Cheat sheet

Posit's [tidytext cheat sheet](#), one page, covers tokenisation, sentiment, tf-idf. Keep it open during the event.

Keep it in scope

Text mining is a means to an analytical insight, not the insight itself. A clear tf-idf or sentiment analysis tied to a specific question scores better than a kitchen-sink pipeline with no narrative tying it together.